ЛР 3. Постоянное хранение и транзакции MySQL

Дисциплина:«Протоколы и интерфейсы информационных систем».

> 💡 Что вы сделаете в лабораторной работе
>
> **ЧТО ВЫ СДЕЛАЕТЕ В ЭТОЙ ЛАБОРАТОРНОЙ**
> • опишите реляционную схему equipment → tickets → ticket_events;
> • подготовите демонстрационные данные и отдельного пользователя приложения;
> • реализуйте MysqlRepository и транзакции commit/rollback;
> • переключите Flask с memory на MySQL и проверить сохранение после перезапуска;
> • запустите интеграционные тесты на отдельной тестовой базе.

1. Перед практикой минимум теории

Таблица хранит объекты одного типа. Первичный ключ идентифицирует строку, внешний ключ связывает её с другой таблицей. У оборудования может быть много заявок, у заявки — много событий. Название оборудования не дублируем в каждой заявке: получаем его через JOIN.

```
SELECT t.id, t.title, e.name AS equipment_name
FROM tickets AS t
JOIN equipment AS e ON e.id = t.equipment_id;
```

SQL использует *snake_case*, JSON — *camelCase*. Метод *_ticket* переводит строку SQL в JSON-модель. Это граница интерфейсов: смена хранения не должна менять ответы */api/tickets.*

В запросах с пользовательскими значениями применяйте параметры *Connector/Python:*

```
cursor.execute("SELECT * FROM tickets WHERE id = %s", (ticket_id,))
```

%s здесь — место значения, передаваемого отдельно, а не форматирование строки Python. Имена таблиц не передаются такими параметрами. Нельзя собирать SQL из пользовательского текста.

Транзакция объединяет операции: *commit* подтверждает все изменения, *rollback* отменяет их при ошибке. Создание заявки и события «created» должно либо выполниться целиком, либо не оставить частичных данных. Курсор и соединение освобождаются даже при исключении.

2. Подготовка проекта и среды MySQL

Активируйте ранее созданное окружение:

```
# macOS / Linux
source backend/.venv/bin/activate

# Windows PowerShell
.\backend\.venv\Scripts\Activate.ps1
```

Если окружение было удалено: *python -m venv backend/.venv,* активируйте его и выполните *python -m pip install -r backend/requirements-dev.txt*. *Connector/Python* уже входит в зависимости.

Проверьте:

```
python -c "import mysql.connector; print(mysql.connector.__version__)".
```

Установите *MySQL Community Server 8.4 LTS* по официальной инструкции для своей ОС:

Windows: загрузите MSI MySQL 8.4 и завершите MySQL Configurator; включите локальный сервер, задайте пароль root, запомните порт (обычно 3306). Добавьте каталог bin MySQL в PATH.

macOS с Homebrew: brew install mysql@8.4, затем brew services start mysql@8.4.
Если mysql не найден, используйте путь, который показывает brew info mysql@8.4.

Linux: подключите официальный MySQL APT Repository для поддерживаемого дистрибутива, выберите серию 8.4 LTS, затем выполните sudo apt update и sudo apt install mysql-server.
Параметры доступа администратора зависят от выбранного способа установки.

В отдельном терминале проверьте mysql --version и вход mysql -h 127.0.0.1 -u root -p.
В приглашении MySQL выполните SELECT VERSION();, затем exit. Пароль вводится интерактивно.
Если используется лабораторный сервер, преподаватель выдаёт адрес, порт и отдельную базу; не запускайте инициализацию на общей базе группы.

3. Практика схема и данные

3.1. Опишите таблицы

> ! Практическое задание
>
> Выполните действия этого подраздела последовательно.
>
> Перед переходом к следующему шагу проверьте ожидаемый результат, указанный в тексте.

Откройте *database/schema.sql*. Замените TODO следующим определением. Прочитайте каждый ограничитель и объясните, почему нельзя удалить оборудование с существующей заявкой, а события удаляются вместе с заявкой.

```
# database/schema.sql
-- MySQL 8.4+, InnoDB. Run against an explicitly selected empty database.
CREATE TABLE IF NOT EXISTS equipment (
  id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(120) NOT NULL,
  inventory_number VARCHAR(40) NOT NULL UNIQUE,
  location VARCHAR(120) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE IF NOT EXISTS tickets (
  id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  equipment_id INT NOT NULL,
  title VARCHAR(120) NOT NULL,
  description TEXT NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'new',
  created_at DATETIME(6) NOT NULL DEFAULT (UTC_TIMESTAMP(6)),
  updated_at DATETIME(6) NOT NULL DEFAULT (UTC_TIMESTAMP(6)),
  CONSTRAINT fk_tickets_equipment FOREIGN KEY (equipment_id) REFERENCES equipment(id),
  CONSTRAINT ck_ticket_status CHECK (status IN ('new','in_progress','closed')),
  CONSTRAINT ck_ticket_title CHECK (CHAR_LENGTH(TRIM(title)) BETWEEN 3 AND 120),
  CONSTRAINT ck_ticket_description CHECK (CHAR_LENGTH(description) <= 2000),
  INDEX ix_tickets_status_created (status, created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE IF NOT EXISTS ticket_events (
  id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  ticket_id INT NOT NULL,
  action VARCHAR(20) NOT NULL,
  message VARCHAR(255) NOT NULL,
  created_at DATETIME(6) NOT NULL DEFAULT (UTC_TIMESTAMP(6)),
  CONSTRAINT fk_events_ticket FOREIGN KEY (ticket_id) REFERENCES tickets(id) ON DELETE
CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Проверьте соответствие: *equipment_id* связан с *equipment.id*; *ticket_id* — с *tickets.id*.
Индексы ускоряют подходящие выборки, но требуют места и дополнительной работы при записи.
Обязательный внешний ключ не заменяет понятную проверку на уровне API.

3.2. Подготовьте демонстрационные данные

> **!** Практическое задание
>
> Выполните действия этого подраздела последовательно.
>
> Перед переходом к следующему шагу проверьте ожидаемый результат, указанный в тексте.

Замените содержимое *database/seed.sql* следующим набором. Эти идентификаторы используются приёмочными тестами: не меняйте их до прохождения проверок. Свои примеры добавьте после сдачи.

```
# database/seed.sql
-- Initial demo data. Execute once after schema.sql in a new database.
INSERT INTO equipment(id, name, inventory_number, location) VALUES
(101, 'Рабочая станция Lenovo ThinkCentre', 'PC-0204-01', 'Аудитория 204'),
(102, 'Проектор Epson EB-X49', 'PR-0301-01', 'Аудитория 301'),
(103, 'Коммутатор TP-Link TL-SG1024', 'NET-0101-01', 'Серверная'),
(104, 'Принтер HP LaserJet Pro', 'PRN-0200-01', 'Кабинет ИТ-отдела');
INSERT INTO tickets(id, equipment_id, title, description, status) VALUES
(1,101,'Не включается компьютер в аудитории 204','После нажатия кнопки питания индикаторы не
горят.','new'),
(2,102,'Проектор не выводит изображение','Кабель подключён, источник сигнала
выбран.','in_progress'),
(3,103,'Восстановить подключение к локальной сети','Подключение восстановлено после замены
кабеля.','closed');
INSERT INTO ticket_events(ticket_id, action, message) VALUES
(1,'created','Учебная заявка загружена'),(2,'created','Учебная заявка загружена'),
(3,'created','Учебная заявка загружена');
```

3.3. Создайте новую базу и пользователя приложения

> ! Практическое задание
>
> Выполните действия этого подраздела последовательно.
>
> Перед переходом к следующему шагу проверьте ожидаемый результат, указанный в тексте.

Из корня проекта выполните:

```
# database/bootstrap.py
python database/bootstrap.py
```

Введите пароль администратора, затем новый пароль приложения длиной не менее 12 символов. Скрипт создаёт *service_desk*, таблицы, данные и *service_desk_app* с правами чтения и изменения данных. Он отказывается перезаписывать существующую базу или пользователя.

Если имена уже заняты, выберите новые:

```
# database/bootstrap.py
python database/bootstrap.py --database service_desk_course --app-user course_app.
```

Для другого порта добавьте *--port НОМЕР*. Далее используйте выбранные имена в *.env*.

Проверьте вход под пользователем приложения:

```
mysql -h 127.0.0.1 -u service_desk_app -p service_desk
```

```
SHOW TABLES;
SELECT COUNT(*) FROM tickets;
SELECT t.id, t.title, e.name FROM tickets t
JOIN equipment e ON e.id=t.equipment_id;
```

Ожидаются три таблицы и три начальные заявки. Сохраните результат JOIN в отчёт.

4. Практика репозиторий и транзакции

4.1. Реализуйте транзакцию

> **!** Практическое задание
>
> Выполните действия этого подраздела последовательно.
>
> Перед переходом к следующему шагу проверьте ожидаемый результат, указанный в тексте.

В *backend/app/repository.py* найдите класс *MysqlRepository*. Соединение, преобразование записи, проверка оборудования и запись события уже подготовлены. Прочитайте их перед правками. Заменяйте только соответствующие методы с TODO. У фрагментов ниже отсутствует внешний отступ класса: вставляйте их с отступом четыре пробела; не дублируйте декоратор *@contextmanager*.

```
# backend/app/repository.py
@contextmanager
def transaction(self):
    connection = mysql.connector.connect(**self.config, connection_timeout=5,
                                         autocommit=False, time_zone="+00:00")
    cursor = connection.cursor(dictionary=True, buffered=True)
    try:
        yield cursor
        connection.commit()
    except Exception:
        connection.rollback()
        raise
    finally:
        cursor.close()
        connection.close()
```

Проследите три пути: нормальный выход, ошибка SQL, исключение проверки. Во всех случаях соединение должно закрыться, при ошибке незавершённые изменения откатываются.

4.2. Реализуйте чтение и фильтрацию

> **!** Практическое задание
>
> Выполните действия этого подраздела последовательно.
>
> Перед переходом к следующему шагу проверьте ожидаемый результат, указанный в тексте.

Замените *list*. Обратите внимание на сборку условий из фиксированных фрагментов SQL и передачу значений отдельным списком. Поиск использует LIKE, фильтры — параметры API из ЛР №2.

```
def list(self, status="", scope="all", q=""):
    conditions, params = [], []
    if status:
        conditions.append("t.status = %s")
        params.append(status)
    if scope == "open":
        conditions.append("t.status <> %s")
        params.append("closed")
    if q:
        conditions.append("t.title LIKE %s ESCAPE '='")
        literal = q.replace("=", "==").replace("%", "=%").replace("_", "=_")
        params.append("%" + literal + "%")
    sql = SELECT_TICKET
    if conditions:
        sql += " WHERE " + " AND ".join(conditions)
    sql += " ORDER BY t.created_at DESC, t.id DESC"
    with self.transaction() as cur:
        cur.execute(sql, tuple(params))
        return [serialize(row) for row in cur.fetchall()]
```

Метод get уже подготовлен. Найдите JOIN и преобразование дат. Почему SELECT не должен возвращать внутренние имена столбцов прямо в JSON? Запишите ответ.

4.3. Реализуйте создание, изменение и удаление

> **!** Практическое задание
>
> Выполните действия этого подраздела последовательно.
>
> Перед переходом к следующему шагу проверьте ожидаемый результат, указанный в тексте.

Последовательно замените три метода. После каждого сверьте аргументы с вызовом в *routes.py*.

create

```
def create(self, values):
    with self.transaction() as cur:
        self._equipment(cur, values["equipmentId"])
        cur.execute("""INSERT INTO tickets(equipment_id,title,description,status)
                       VALUES (%s,%s,%s,%s)""",
                    (values["equipmentId"], values["title"], values["description"], "new"))
        ticket_id = cur.lastrowid
        self._record(cur, ticket_id, "created", "Заявка зарегистрирована")
        return self._ticket(cur, ticket_id)
```

update

```
def update(self, ticket_id, values):
    with self.transaction() as cur:
        # Lock the main row before reading old values and writing the event.
        cur.execute("SELECT id FROM tickets WHERE id=%s FOR UPDATE", (ticket_id,))
        if cur.fetchone() is None:
            raise ApiError("NOT_FOUND", "Заявка не найдена.", 404)
        old = self._ticket(cur, ticket_id)
        if "equipmentId" in values:
            self._equipment(cur, values["equipmentId"])
        columns = {"equipmentId": "equipment_id", "title": "title",
                   "description": "description", "status": "status"}
        # Identifiers come only from this fixed mapping; values remain parameters.
        assignments = ", ".join(columns[key] + "=%s" for key in values)
        cur.execute("UPDATE tickets SET " + assignments + ", updated_at=UTC_TIMESTAMP(6)
WHERE id=%s",
                    tuple(values.values()) + (ticket_id,))
        message = "Обновлены поля: " + ", ".join(values)
        if "status" in values and old["status"] != values["status"]:
            labels = {"new": "Новая", "in_progress": "В работе", "closed": "Закрыта"}
            message = f"Статус: {labels[old['status']]} → {labels[values['status']]}"
        self._record(cur, ticket_id, "updated", message)
        return self._ticket(cur, ticket_id)
```

delete

```
def delete(self, ticket_id):
    with self.transaction() as cur:
        cur.execute("DELETE FROM tickets WHERE id=%s", (ticket_id,))
```

```
        if cur.rowcount == 0:
            raise ApiError("NOT_FOUND", "Заявка не найдена.", 404)
```

После создания используется новый идентификатор БД, а не вычисление max(id)+1 в Python. История записывается в той же транзакции, что и заявка. Проверьте, что код не выполняет ранний commit между INSERT заявки и INSERT события.

4.4. Переключите приложение на MySQL

Измените *backend/.env* (файл создан в ЛР №1). Укажите свой пароль и выбранные имена:

```
# backend/.env
STORAGE=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_NAME=service_desk
DB_USER=service_desk_app
DB_PASSWORD=ваш_пароль
```

Остановите Flask сочетанием Ctrl+C. Из каталога backend запустите:

```
python -m flask --app app run --debug --port 5000
```

Во втором терминале из корня с активированным окружением выполните:

```
python tools/request.py GET /api/health
python tools/request.py GET /api/tickets
python tools/request.py POST /api/tickets --data requests/create-ticket.json
```

Health должен сообщить storage: mysql. Запомните ID новой заявки, перезапустите Flask и получите /api/tickets/ID: запись должна остаться. Заменяйте ID реальным числом без скобок.
Проверьте PATCH и DELETE с помощью примеров ЛР №2. Не удаляйте начальные данные до проверки UI.

5. Проверки

Тесты пересоздают таблицы. Создайте отдельную базу, имя которой оканчивается на _test. Следующий SQL выполняет администратор MySQL; замените пароль своим тестовым паролем:

```
CREATE DATABASE service_desk_course_test CHARACTER SET utf8mb4;
CREATE USER 'course_test'@'localhost' IDENTIFIED BY 'ReplaceThisTestPassword123!';
```

```
GRANT ALL PRIVILEGES ON service_desk_course_test.* TO 'course_test'@'localhost';
```

В терминале из корня задайте параметры. Они относятся только к тестовой базе:

```
# macOS / Linux
export TEST_DB_NAME=service_desk_course_test
export TEST_DB_HOST=127.0.0.1
export TEST_DB_PORT=3306
export TEST_DB_USER=course_test
export TEST_DB_PASSWORD='ReplaceThisTestPassword123!'
# Windows PowerShell
$env:TEST_DB_NAME='service_desk_course_test'
$env:TEST_DB_HOST='127.0.0.1'
$env:TEST_DB_PORT='3306'
$env:TEST_DB_USER='course_test'
$env:TEST_DB_PASSWORD='ReplaceThisTestPassword123!'
python tools/require_test_db.py
cd backend
python -m pytest -q
```

Ожидается успешное прохождение всех проверок, включая test_api_mysql.py. Пропуски SQL-тестов не считаются проверкой MySQL. Если переменная не задана, pytest пропускает их, поэтому отдельно запускается проверка имени базы. Возвратитесь в корень: cd ...

Откройте тест отката в test_api_mysql.py: он имитирует сбой записи события и проверяет, что заявка не сохранилась. Поставьте точку останова перед commit и пройдите тест в отладчике.

6. Самостоятельная часть

1. Напишите SELECT количества заявок по каждому статусу с GROUP BY. Сравните с /api/stats.
2. Отправьте заявку с несуществующим equipmentId. Докажите, что количество строк не изменилось.
3. Объясните отличие прав пользователя приложения и пользователя тестов.
4. Добавьте собственный тест фильтрации по статусу и строке поиска одновременно; ожидаемый набор вычислите вручную по seed, а не повторением реализации запроса в тесте.

7. Затруднения и способы проверки

Connection refused: сервер не запущен или отличается порт.

Access denied: проверьте пользователя, пароль и host; приложение не должно работать от root.

Unknown database/table: проверьте имя и выполнение schema/seed; повторный bootstrap не ремонтирует базу.

Ответ 501: остался метод TODO в MySQL-репозитории.

После перезапуска исчезают данные: проверьте STORAGE и health, затем перезапустите Flask из backend.

Тесты skipped: задайте TEST_DB_NAME; зелёный вывод с пропусками не доказывает интеграцию.

8. Что сдавать

> 💡 Перечень
>
> • актуальный проект it-service-desk с выполненной лабораторной;
>
> • результаты предусмотренных в методичке проверок/тестов;
>
> • демонстрацию ожидаемого пользовательского или серверного сценария;
>
> • ответы на вопросы и результаты.

9. Материалы для подготовки

1. MySQL 8.4 Tutorial.
2. MySQL 8.4 Windows installation.
3. MySQL APT Repository
4. Homebrew mysql@8.4.
5. Connector/Python execute.
6. Connector/Python rollback.